Foundry Cheatcodes for Smart Contract Auditing

Core Cheatcodes

- vm.prank
- vm.startPrank
- vm.stopPrank
- vm.deal
- vm.expectRevert
- vm.expectEmit
- vm.warp
- vm.roll

Fuzzing Cheatcodes

- vm.assume
- vm.bound

External Call Control

- vm.expectCall
- vm.mockCall

Storage & Low-Level (Advanced)

- vm.store
- vm.load

Logs & Debugging

- vm.recordLogs
- vm.getRecordedLogs

State Management

- vm.snapshot
- vm.revertTo

Cryptography

- vm.sign
- vm.addr

Advanced / Elite

- vm.etch

Scripts / Deployment

- vm.startBroadcast
- vm.stopBroadcast
- vm.envUint
- vm.envAddress
- vm.envBytes32
- Vm.envString

CHEATCODE EXAMPLES:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";

contract Dummy {
   uint256 public value;
   address public owner;

   constructor() {
      owner = msg.sender;
   }

   function setValue(uint256 _v) public {
      value = _v;
   }

   function onlyOwner() public view {
      require(msg.sender == owner, "NotOwner");
   }

   function sendEther(address payable to) public payable {
      to.transfer(msg.value);
```

```
    }
}

contract CheatcodesTest is Test {
    Dummy dummy;

    address user = address(1);
    address attacker = address(2);

    function setUp() public {
        dummy = new Dummy();
    }

    // ========================
            🔥 CORE CHEATCODES
    // ========================

    function test_prank() public {
        vm.prank(user);
        dummy.setValue(10);

        assertEq(dummy.value(), 10);
    }

    function test_startPrank() public {
        vm.startPrank(user);
        dummy.setValue(1);
        dummy.setValue(2);
        vm.stopPrank();

        assertEq(dummy.value(), 2);
    }


    function test_deal() public {
        vm.deal(user, 10 ether);
        assertEq(user.balance, 10 ether);
    }


    function test_expectRevert() public {
        vm.expectRevert("NotOwner");
        vm.prank(user);
```

```
    dummy.onlyOwner();
}

function test_warp() public {
    uint256 future = block.timestamp + 1 days;
    vm.warp(future);

    assertEq(block.timestamp, future);
}


function test_roll() public {
    vm.roll(block.number + 10);
    assertEq(block.number, 11);
}

function test_expectEmit() public {
    vm.expectEmit(false, false, false, false);
    emit TestEvent(1);

    emit TestEvent(1);
}

event TestEvent(uint256 val);




// ==========================
    ⚔️ FUZZING CHEATCODES
// ==========================

function test_assume(uint256 x) public {
    vm.assume(x > 10);
    assertTrue(x > 10);
}

function test_bound(uint256 x) public {
    x = bound(x, 1, 100);
    assertTrue(x >= 1 && x <= 100);
}
```

```
// ==========================
   🧪 EXTERNAL CALL CONTROL
// ==========================

function test_expectCall() public {
   bytes memory data = abi.encodeWithSelector(dummy.setValue.selector, 5);

   vm.expectCall(address(dummy), data);
   dummy.setValue(5);
}

function test_mockCall() public {
   address target = address(100);

   vm.mockCall(
      target,
      abi.encodeWithSignature("foo()"),
      abi.encode(999)
   );

   (bool success, bytes memory data) =
      target.call(abi.encodeWithSignature("foo()"));

   uint256 result = abi.decode(data, (uint256));
   assertEq(result, 999);
}



// ==========================
// 🔍 STORAGE (ELITE)
// ==========================

function test_store_and_load() public {
   // slot 0 = value
   vm.store(address(dummy), bytes32(uint256(0)), bytes32(uint256(777)));

   uint256 val = dummy.value();
```

```
    assertEq(val, 777);

    bytes32 raw = vm.load(address(dummy), bytes32(uint256(0)));
    assertEq(uint256(raw), 777);
}




// ==========================
// 📜 LOG INSPECTION
// ==========================

function test_recordLogs() public {
    vm.recordLogs();

    dummy.setValue(42);

    Vm.Log[] memory logs = vm.getRecordedLogs();

    assertGt(logs.length, 0);
}




// ==========================
// 🔁 SNAPSHOT
// ==========================

function test_snapshot() public {
    uint256 snap = vm.snapshot();

    dummy.setValue(100);
    assertEq(dummy.value(), 100);

    vm.revertTo(snap);
    assertEq(dummy.value(), 0);
}
```

```
    // ========================
    // 🔐 SIGNATURES
    // ========================

    function test_sign() public {
        uint256 pk = 1;
        address signer = vm.addr(pk);

        bytes32 digest = keccak256("hello");

        (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, digest);

        address recovered = ecrecover(digest, v, r, s);

        assertEq(recovered, signer);
    }

    // ========================
    // 🧱 ADVANCED: ETCH
    // ========================

    function test_etch() public {
        address target = address(200);

        bytes memory code = address(dummy).code;

        vm.etch(target, code);

        // now target behaves like dummy contract
        Dummy(target).setValue(55);

        assertEq(Dummy(target).value(), 55);
    }
}
```